



Universidade Federal de Sergipe
Departamento de Computação
Engenharia de Software



ATIVIDADE AVALIATIVA 3
AUDITORIA DE TESTES DE SOFTWARE

Projeto Analisado – anythingLLM

Repositório do Projeto – [Clique Aqui](#)

Repositório da Auditoria – [Clique Aqui](#)

Vídeo da Análise – [Clique Aqui](#)

Ariel Santos Souza – 202400050731

Contribuição: Estratégia Atual dos Testes

Emilly Kathellen de Jesus Oliveira – 202400050830

Contribuição: Lacunas, Riscos e Falhas Potenciais

Giovani Barros Luna Gomes – 202400050885

Contribuição: Lacunas, Riscos e Falhas Potenciais

Karol Sthefanny da Cruz Menezes – 202400050947

Contribuição: Qualidade Técnica dos Testes

Nycolly Souza Sena – 202400051004

Contribuição: Estratégia Atual dos Testes

Sara Cristina Santos Chagas – 202400029077

Contribuição: Qualidade Técnica dos Testes

Orientador – Glauco de Figueiredo Carneiro

Abril de 2026

Sumário

1. INTRODUÇÃO.....	2
2. METODOLOGIA.....	3
3. ESTRATÉGIA ATUAL DOS TESTES.....	4
3.1. Evidências.....	4
3.2. Diagnóstico.....	16
3.3. Risco.....	17
3.4. Recomendação.....	17
4. QUALIDADE TÉCNICA DOS TESTES.....	19
4.1. Visão geral da suíte de testes.....	19
4.2. Evidências.....	19
4.3. Diagnóstico.....	22
4.4. Risco.....	23
4.5. Recomendação.....	26
5. LACUNAS, RISCOS E FALHAS POTENCIAIS.....	28
5.1. Evidências.....	28
5.2. Diagnóstico.....	36
5.3. Risco.....	38
5.4. Recomendação.....	40
6. PLANO DE EVOLUÇÃO DA QUALIDADE.....	43
6.1. Top 3 prioridades de teste.....	43
6.2. Camadas de foco.....	43
6.3. Implantação gradual.....	44
6.4. Ferramentas, pipeline e política mínima de execução.....	44
6.5. Critérios de sucesso.....	45
7. REFERÊNCIAS.....	46

1. INTRODUÇÃO

Este relatório tem como objetivo central realizar um diagnóstico da estratégia de testes de software do projeto AnythingLLM, disponível publicamente no GitHub, investigando a existência, a qualidade, a cobertura e a relevância dos testes automatizados adotados pela equipe mantenedora. A partir da análise da estrutura do repositório, dos arquivos de configuração, dos pipelines de Integração Contínua e do histórico de Pull Requests e Issues, busca-se identificar riscos, lacunas e fragilidades presentes na atual abordagem de qualidade de software do projeto. Por fim, com base nesse diagnóstico, a equipe propõe um plano concreto e priorizado de evolução da qualidade do produto, fundamentado em boas práticas de teste reconhecidas pelo mercado, de modo a orientar futuras contribuições e fortalecer a confiabilidade do sistema frente à sua complexidade arquitetural e à sua superfície crescente de integrações externas.

2. METODOLOGIA

A auditoria foi conduzida a partir da análise direta do repositório oficial do AnythingLLM no GitHub, combinando inspeção do código-fonte, dos arquivos de configuração (package.json, workflows do GitHub Actions), do histórico de commits, Pull Requests e Issues, além de documentação complementar disponibilizada pelo próprio projeto (como o CONTRIBUTING.md) e por ferramentas de mapeamento de código como o DeepWiki. Para cada aspecto investigado, a equipe adotou um framework de análise estruturado em quatro etapas consistentes, aplicadas uniformemente em todas as seções do relatório:

- **Evidência:** coleta de elementos objetivos e verificáveis que sustentam a análise, como prints de tela, links diretos para arquivos ou linhas de código no repositório, trechos de código-fonte, logs de execução de pipelines, arquivos de configuração de CI/CD e registros de Issues públicas;
- **Diagnóstico:** leitura crítica da equipe sobre o que as evidências coletadas efetivamente revelam, contextualizando os achados dentro da arquitetura e da proposta de valor do sistema;
- **Risco:** classificação do achado em nível Baixo, Médio ou Alto, sempre acompanhada de justificativa técnica que considera a criticidade da funcionalidade afetada, a probabilidade de regressão e o impacto potencial sobre usuários e mantenedores;
- **Recomendação:** proposta objetiva e acionável de melhoria, priorizada de acordo com a relação entre esforço de implementação e redução de risco.

Essa metodologia foi aplicada às três frentes de investigação que estruturam o corpo do relatório: a Estratégia Atual dos Testes (organização, ferramentas e infraestrutura de CI/CD), a Qualidade Técnica dos Testes (análise aprofundada de um arquivo de teste representativo quanto a nomenclatura, isolamento, reprodutibilidade e uso de mocks/fixtures) e o levantamento de Lacunas, Riscos e Falhas Potenciais (cruzamento entre a cobertura de testes existente e os pontos de maior criticidade arquitetural identificados em atividades anteriores da disciplina).

3. ESTRATÉGIA ATUAL DOS TESTES

3.1. Evidências

3.1.1. Pasta dedicada a testes (`__tests__`)

O server e collector possuem uma pasta `__tests__`, o que indica uma convenção explícita para organização dos testes automatizados do back-end. Toda a suíte de testes baseia-se fortemente no framework **Jest**, utilizando intensamente o recurso de **Mocks** (`jest.mock`), por meio de arquivos `.test.js`. Essa estratégia isola o código sob teste, simulando o banco de dados (Prisma), variáveis de ambiente e conexões de rede para validar as funções de forma pura e previsível.

→ [server/ __tests__](#)

→ [collector/ __tests__ /utils](#)

Dentro do diretório de testes de server, a subpasta `/models` é dedicada a validar as regras de negócio, as validações de dados e os comportamentos centrais das entidades do sistema (os "modelos"). A análise dos testes contidos nessa pasta revela como a aplicação assegura a consistência de seus principais componentes:

Gerenciamento de Filas e Configurações: Valida o comportamento da fila de sincronização de documentos (`DocumentSyncQueue`). Os testes focam em garantir que o sistema possua fallback seguro para variáveis de ambiente. Se a variável `DOCUMENT_SYNC_STALE_AFTER_MS` estiver ausente, inválida (não numérica) ou com valores negativos, o sistema assume com segurança o padrão de 7 dias (604.800.000 ms). Além disso, garante que valores muito baixos sejam limitados ao mínimo seguro de 1 hora (3.600.000 ms).

Processamento e Interpolação de Prompts: Testa a capacidade do motor do AnythingLLM de expandir variáveis dinâmicas inseridas nos prompts do sistema (como `{workspace.name}`, `{user.name}` ou variáveis estáticas personalizadas). Os testes utilizam simulações (*mocks*) do banco de dados (Prisma) para checar se a substituição de strings ocorre perfeitamente em múltiplos cenários e garantem que o sistema falhe de forma elegante (*fail gracefully*), substituindo propriedades inválidas ou confidenciais (como senhas) por marcadores genéricos (ex: `[User password]`), sem quebrar a execução.

Regras de Validação de Usuários: Concentra-se em restringir e sanitizar a criação de nomes de usuário (username). Assegura, via tratamento de exceções (*errors*), que os nomes de usuário obedeçam estritamente aos critérios do sistema: ter entre 2 e 32 caracteres, começar obrigatoriamente com letra minúscula e conter apenas caracteres permitidos (letras minúsculas, números, sublinhados, hífen e pontos), abrindo exceção válida para formatos de e-mail.

Mocks para testes

```
const { SystemPromptVariables } = require("../models/systemPromptVariables");
const prisma = require("../utils/prisma");

const mockUser = {
  id: 1,
  username: "john.doe",
  bio: "I am a test user",
};

const mockWorkspace = {
  id: 1,
  name: "Test Workspace",
  slug: 'test-workspace',
};

// ...

describe("SystemPromptVariables.expandSystemPromptVariables", () => {
  beforeEach(() => {
    jest.clearAllMocks();
    prisma.system_prompt_variables.findMany =
    jest.fn().mockResolvedValue(mockSystemPromptVariables);
    prisma.workspaces.findUnique = jest.fn().mockResolvedValue(mockWorkspace);
    prisma.users.findUnique = jest.fn().mockResolvedValue(mockUser);
  });

  // ...
});
```

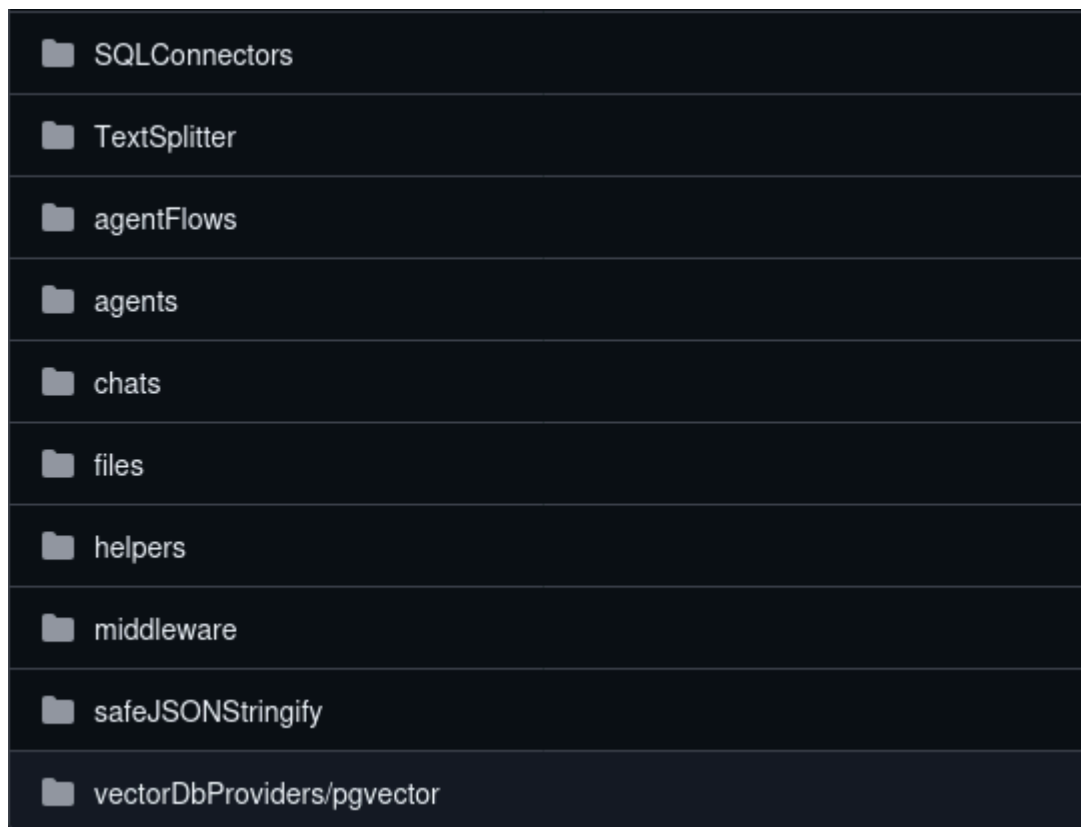
[server/_tests_/models/systemPromptVariables.test.js](#)

Já a subpasta `/utils` concentra os testes das ferramentas utilitárias, integradores de infraestrutura e motores de execução do servidor.

Os subdiretórios contidos em /utils organizam-se da seguinte forma:

- **SQLConnectors:** Valida a camada de conexão, execução de queries e sanitização de dados com diferentes bancos de dados relacionais suportados pela aplicação.
- **TextSplitter:** Testar os algoritmos de quebra de texto (tokenização e chunking). Garante que documentos longos sejam fragmentados corretamente antes de serem enviados aos modelos de incorporação (*embeddings*).
- **agentFlows:** Garante a integridade dos fluxos de execução sequenciais ou paralelos que coordenam múltiplos agentes e automações complexas dentro do sistema.
- **agents:** Dedicada a testar o comportamento e as capacidades dos agentes de IA do AnythingLLM. Os testes nesta pasta cobrem:
- **chats:** Analisa as funções de manipulação de histórico, formatação de mensagens, contagem de tokens e gerenciamento de sessões de conversação.
- **files:** Valida utilitários de manipulação de arquivos locais, upload, download, checagem de extensões permitidas e isolamento de diretórios de armazenamento.
- **helpers:** Reúne testes de funções acessórias genéricas reutilizadas por todo o ecossistema do servidor (manipulação de strings, datas e arrays).
- **middleware:** Garante o correto funcionamento das barreiras de segurança nas requisições HTTP, como autenticação de tokens, controle de acessos (RBAC) e limites de requisição (*rate limiting*).
- **safeJSONStringify:** Valida o utilitário responsável por serializar objetos em formato JSON de maneira segura, mitigando falhas clássicas do ecossistema Node.js como referências circulares ou estouro de memória (*call stack overflow*).
- **vectorDbProviders/pgvector:** Concentra os testes específicos de integração com o ecossistema de bancos de dados vetoriais, simulando e validando buscas por similaridade, inserção de embeddings e queries vetoriais nativas no PostgreSQL (pgvector).

Subdiretórios de `__tests__`/utils



[server/ __tests__ /utils](#)

Dessa forma, a pasta `/utils` reflete o cuidado do projeto em blindar a infraestrutura e os canais de integração externa, assegurando que o processamento pesado e o ecossistema de agentes operem sob comportamentos rigidamente testados.

3.1.2. Script de teste no `package.json` raiz

O projeto AnythingLLM adota uma estrutura de monorepo (dividida em `server`, `frontend` e `collector`), mas centraliza a orquestração de testes na raiz da aplicação. A análise do arquivo `package.json` raiz revela como o ecossistema de testes é configurado e disparado:

Centralização com o Jest: No bloco `"scripts"`, o comando `"test": "jest"` estabelece o Jest como a ferramenta padrão e unificada de testes. Isso permite que qualquer desenvolvedor ou pipeline de automação execute toda a suíte de testes do projeto simplesmente rodando `yarn test` ou `npm test` a partir do diretório raiz, sem a necessidade de navegar manualmente pelas subpastas do servidor.

Dependência de Desenvolvimento Atualizada: No bloco "devDependencies", identifica-se a declaração "jest": "^29.7.0". A escolha da versão 29 (uma das mais maduras e recentes do framework) garante ao projeto recursos avançados de performance (como execução paralela de testes usando *workers*), suporte robusto a simulações complexas (*mocking*) e compatibilidade nativa com sintaxes modernas do Node.js.

Facilitação para Integração Contínua (CI/CD): A existência de um comando único e previsível na raiz simplifica drasticamente a configuração de esteiras de automação (como GitHub Actions). O servidor de CI precisa apenas clonar o repositório, rodar o script de setup e disparar o comando de teste para validar a integridade de todo o ecossistema antes de permitir um *deploy*.

Comando para testes com Jest

```
"scripts": {
  "test": "jest",
// ...
  "dev:collector": "cd collector && yarn dev",
  "dev:frontend": "cd frontend && yarn dev",
  "dev": "npx concurrently \"yarn dev:server\" \"yarn dev:frontend\" \"yarn dev:collector\"",
// ...
},
```

[/package.json](#)

O package.json específico do servidor (server/package.json) e collector (server/package.json) não define nenhum script test, contendo apenas scripts de desenvolvimento (dev, start, lint, swagger). Isso significa que os testes do servidor são executados somente a partir da raiz do monorepo, e não de forma independente pelo submódulo.

3.1.3. Workflow de CI "Run backend tests" no GitHub Actions

O projeto possui um workflow de GitHub Actions chamado "**Run backend tests**", disparado automaticamente em Pull Requests. Ele aparece consistentemente nos registros de execução da aba Actions do repositório, confirmando integração com CI/CD para execução de testes no back-end a cada contribuição externa.

 <https://github.com/Mintplex-Labs/anything-llm/actions>

O projeto possui um workflow de GitHub Actions chamado "Run backend tests", disparado automaticamente em Pull Requests. Ele aparece consistentemente nos registros de execução da aba Actions do repositório, confirmando integração com CI/CD para execução de testes no back-end a cada contribuição externa.

Uma análise detalhada do arquivo de configuração `run-tests.yaml` revela uma estratégia de esteira de integração contínua altamente otimizada e segura:

Gatilhos Inteligentes e Escopo de Arquivos (on): O workflow não roda de maneira indiscriminada. Ele é ativado apenas em Pull Requests (abertura, sincronização ou reabertura) e possui filtros de caminhos (paths). A esteira só é disparada se houver modificações em arquivos JavaScript (.js) dentro dos diretórios `server/` ou `collector/`, poupando recursos computacionais quando as alterações forem apenas no frontend ou na documentação.

Gerenciamento de Concorrência Eficiente (concurrency): O arquivo configura o cancelamento automático de tarefas obsoletas (`cancel-in-progress: true`) baseando-se no grupo formado pelo nome do workflow e a branch de origem (`github.ref`). Caso um desenvolvedor envie um novo *commit* para um Pull Request enquanto o teste anterior ainda estiver rodando, a execução antiga é interrompida imediatamente para priorizar o código mais recente, otimizando o tempo de fila da infraestrutura.

Ambiente de Execução Padronizado: O job `run-script` é executado sobre a imagem mais recente do Ubuntu (`ubuntu-latest`) e utiliza a action oficial `actions/setup-node@v3` para fixar o ambiente no **Node.js versão 18**. Isso garante total paridade entre o ambiente de testes do servidor de CI e as diretrizes do projeto (`"engines": { "node": ">=18" }` vistas no `package.json` raiz).

Estratégia Agressiva de Cache (actions/cache): Para acelerar o tempo total da esteira, o workflow cria de forma isolada três estruturas de cache de dependências de pacotes:

- Raiz do projeto (`node_modules`).
- Subprojeto do servidor (`server/node_modules`).
- Subprojeto do coletor de documentos (`collector/node_modules`).

Os caches são atrelados à assinatura hash dos respectivos arquivos yarn.lock. Se os arquivos de dependência não mudarem, o download é ignorado nas próximas execuções.

Instalação Determinística com Otimizações: As dependências são instaladas através do comando `yarn install --frozen-lockfile`, garantindo que o servidor de CI utilize exatamente as mesmas versões salvas no repositório, impedindo atualizações silenciosas que poderiam quebrar o código. Na instalação do coletor, são passadas flags para pular downloads pesados e desnecessários para a rotina de testes (como o binário do Chromium via `PUPPETEER_SKIP_DOWNLOAD: "true"`).

Inicialização e Disparo: Antes de rodar os testes, a esteira executa os scripts nativos do projeto para clonar os arquivos de variáveis de ambiente de exemplo (`yarn setup:envs`) e inicializar a geração de tipos e migrações do banco de dados relacional (`yarn prisma:setup`). Somente após essa infraestrutura estar pronta, o comando principal "test": "jest" é disparado globalmente através do `yarn test`.

Garantia de Qualidade (*Fail-Safe*): O passo final utiliza a condicional `if: failure()` para forçar um encerramento explícito com erro (exit 1) caso qualquer suite ou expectativa do Jest falhe. Isso bloqueia automaticamente a possibilidade de fusão (*merge*) do Pull Request no GitHub até que o código seja devidamente corrigido pelo desenvolvedor.

Workflow de testes

```
name: Run backend tests

concurrency:
  group: ${ github.workflow }-${ github.ref }
  cancel-in-progress: true

on:
  pull_request:
    types: [opened, synchronize, reopened]
    paths:
      - "server/**/*.js"
      - "collector/**/*.js"

jobs:
  run-script:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v2

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
      // ...
      - name: Install server dependencies
        if: steps.cache-server.outputs.cache-hit != 'true'
        run: cd server && yarn install --frozen-lockfile

      - name: Setup environment and Prisma
        run: yarn setup:envs && yarn prisma:setup

      - name: Run test suites
        run: yarn test

      - name: Fail job on error
        if: failure()
        run: exit 1
```

[.github/workflows/run-tests.yaml](#)

3.1.4. Workflow de lint via CI (ESLint)

O projeto conta com uma esteira automatizada focada na qualidade, padronização estática e formatação do código-fonte, gerenciada pelo GitHub Actions através do workflow "Lint".

Uma análise detalhada do arquivo de configuração `lint.yaml` evidencia práticas consolidadas de governança de código em repositórios multi-módulos (monorepos):

Escopo Abrangente e Segmentado (on.paths): O gatilho de execução é acionado em Pull Requests sempre que houver modificações em arquivos de lógica ou visualizações (`.js` e `.jsx`), abrangendo os três pilares do projeto: `server/`, `collector/` e `frontend/`. Além disso, de forma inteligente, o workflow monitora alterações nos próprios arquivos de configuração do linter (`eslint.config.js` e `.mjs`), garantindo que qualquer mudança nas regras de estilo seja imediatamente testada e validada em toda a base de código.

Controle de Concorrência e Redução de Desperdício (concurrency): Assim como na esteira de testes, o mecanismo de concorrência cancela automaticamente execuções anteriores em andamento na mesma branch de origem (`cancel-in-progress: true`) caso um novo empurrão de código (*push*) aconteça. Isso evita o desperdício de minutos computacionais do GitHub Actions com versões obsoletas do código.

Modernização da Infraestrutura de Actions: Diferente de outros fluxos, este arquivo faz uso de versões atualizadas das dependências oficiais do GitHub, como `actions/checkout@v4` e `actions/setup-node@v4`, capturando correções de bugs nativas, melhorias de segurança e performance no provisionamento do ambiente Node.js 18.

Estratégia de Cache por Subprojeto (actions/cache): O fluxo gerencia três instâncias independentes de cache para os diretórios `node_modules` do servidor, frontend e coletor. As chaves de cache utilizam a função `hashFiles` atrelada a cada arquivo `yarn.lock` específico. Essa granularidade impede que alterações de dependências feitas no frontend invalidem o cache de dependências estáveis do back-end, acelerando consideravelmente o ciclo de feedback para o desenvolvedor.

Instalação Determinística e Otimizada: Respeitando as travas de versão do ecossistema do projeto, a instalação ocorre via `yarn install --frozen-lockfile`. Para

otimizar o tempo no container do coletor, variáveis de ambiente específicas são injetadas (`PUPPETEER_SKIP_DOWNLOAD: "true"`) para mitigar o download de binários pesados de renderização que não possuem impacto na análise estática de código.

Execução Modularizada de Varredura: O coração do job executa de forma sequencial o comando `yarn lint:check` dentro de cada subdiretório. Essa abordagem isolada garante que, caso o código do frontend desrespeite as regras de formatação (como variáveis declaradas e não utilizadas ou quebras de padrão estético), a esteira aponte precisamente onde ocorreu o desvio. Ao mapear o `package.json` raiz visto anteriormente, nota-se que o comando `lint:check` atua como a barreira de validação (*check-only*), ideal para ambientes de CI onde o código não deve ser alterado automaticamente, mas sim rejeitado caso contenha inconformidades.

Em suma, a existência paralela dos workflows de teste e de lint assegura que nenhuma contribuição externa chegue à branch principal se violar os critérios de cobertura lógica ou quebrar a padronização estilística arquitetada para o AnythingLLM.

Workflow de Lint

```
name: Lint

concurrency:
  group: lint-${{ github.ref }}
  cancel-in-progress: true

on:
  pull_request:
    types: [opened, synchronize, reopened]
    paths:
      - "server/**/*.js"
      - "server/eslint.config.mjs"
      - "collector/**/*.js"
      - "collector/eslint.config.mjs"
      - "frontend/src/**/*.js"
      - "frontend/src/**/*.jsx"
      - "frontend/eslint.config.js"

jobs:
  lint:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      // ...
      - name: Install frontend dependencies
        run: cd frontend && yarn install --frozen-lockfile

      - name: Lint server
        run: cd server && yarn lint:check

      - name: Lint frontend
        run: cd frontend && yarn lint:check

      - name: Lint collector
        run: cd collector && yarn lint:check
```

[.github/workflows/lint.yaml](https://github.com/workflows/lint.yaml)

3.1.5. Indicadores de cobertura

A análise do processo de testes do AnythingLLM indica a ausência de mecanismos formais de monitoramento de cobertura de código. Não foram encontrados badges de qualidade, integrações com ferramentas como Codecov ou SonarCloud, nem configurações relacionadas à geração de métricas de cobertura no projeto. As buscas na base de código também não identificaram parâmetros como `--coverage` ou `collectCoverage`, sugerindo que o framework de testes é executado apenas para validar o sucesso ou falha das suítes. Essa constatação foi confirmada pela inspeção dos logs do pipeline de Integração Contínua, nos quais o Jest apresenta apenas o resultado das execuções dos testes, sem exibir relatórios quantitativos de cobertura. Dessa forma, o projeto não possui indicadores que permitam mensurar quais partes do código estão efetivamente protegidas contra regressões e falhas futuras.

Registro visual do sumário de encerramento do passo *Run test suites* coletado diretamente no console de execução do GitHub Actions. O log comprova empiricamente a conclusão estritamente binária da rotina do Jest, que reporta apenas o sucesso das asserções (194 testes passados em 5.85 segundos) e encerra o pipeline sem renderizar nenhuma matriz percentual de cobertura de linhas ou blocos de código.

Run Test Suites

```
132
133 Test Suites: 1 skipped, 22 passed, 22 of 23 total
134 Tests:      5 skipped, 194 passed, 199 total
135 Snapshots:  0 total
136 Time:       5.165 s
137 Ran all test suites.
138 Done in 5.85s.
```

<https://github.com/Mintplex-Labs/anything-llm/actions/runs/28139729928/job/83334271758>

3.2. Diagnóstico

A avaliação holística da engenharia de testes do *AnythingLLM* revela uma estratégia puramente voltada à validação lógica back-end, caracterizada por um forte isolamento de componentes e uma infraestrutura de Integração Contínua (CI/CD) altamente automatizada, porém metricamente cega. A existência de pastas dedicadas (`__tests__`) subdivididas em áreas de negócio (`/models`) e utilitários de infraestrutura (`/utils`) comprova o estabelecimento de uma convenção clara de design de testes dentro do monorepo. O ecossistema apoia-se de forma madura no framework Jest (versão 29.7.0) e utiliza intensamente recursos de simulação (`jest.mock`) para desacoplar o código sob análise de dependências complexas como o ORM Prisma, variáveis de ambiente e requisições HTTP externas. Essa abordagem garante alta previsibilidade e velocidade na validação de utilitários puros (como o `TextSplitter` e sanitizadores de string), cujas *fixtures* de dados simulam com precisão casos limítrofes e recursões de objetos aninhados.

A orquestração desses testes é eficientemente centralizada no `package.json` da raiz do projeto, permitindo o disparo unificado via `yarn test`. Essa centralização estende-se para os pipelines automatizados do GitHub Actions, onde o workflow `Run backend tests` (`run-tests.yaml`) demonstra excelente maturidade de engenharia de confiabilidade. O uso de filtros inteligentes por caminhos de arquivos (`paths`), o cancelamento automático de tarefas obsoletas em andamento (`cancel-in-progress`) e estratégias agressivas de cache triplo baseadas nos arquivos `yarn.lock` evidenciam uma preocupação explícita com a eficiência computacional e o tempo de feedback para os desenvolvedores. Além disso, a presença de uma esteira paralela de análise estática (`lint.yaml`) garante que nenhuma contribuição seja integrada à branch principal caso viole padrões de estilo e formatação definidos pelo ESLint.

Contudo, o principal ponto fraco dessa estratégia reside na completa ausência de métricas quantitativas de qualidade. Conforme evidenciado pela omissão de parâmetros como `--coverage` no script de execução global e corroborado pelos logs reais emitidos pelo pipeline de CI, o Jest atua de forma estritamente booleana (`pass/fail`). Embora o sistema valide com sucesso a execução bem-sucedida de 194 testes lógicos em ambiente de integração, ele encerra as rotinas sem computar ou exibir matrizes de cobertura sobre quais blocos ou linhas de código foram de fato exercitados.

Dentro do escopo mapeado, a infraestrutura de automação de testes e de análise estática apresenta-se altamente otimizada, com dados expressivos e um isolamento de componentes adequado para funções puras do back-end. No entanto, a execução puramente binária e a falta de parametrização estatística apontam para uma grave lacuna estrutural de governança: o ecossistema opera sob total invisibilidade métrica, não fornecendo qualquer evidência quantitativa ou histórica sobre a real cobertura de código e a proteção do sistema contra regressões em módulos críticos.

3.3. Risco

A atual abordagem de testes, apesar de robusta em sua automação de infraestrutura e isolamento de escopo, introduz um risco de nível Médio originado por um ponto cego estatístico. O fato de os scripts existentes passarem com sucesso nas esteiras de CI/CD gera uma falsa sensação de segurança técnica para a equipe de desenvolvimento. Em uma aplicação com a complexidade arquitetural do *AnythingLLM*, que lida diretamente com fluxos de agentes de IA, middlewares de segurança de rotas, parsers de banco de dados relacionais e integrações dinâmicas com múltiplos provedores externos de LLMs, a falta de relatórios de *Code Coverage* impede a identificação de lacunas críticas de teste.

Sem indicadores visíveis ou badges de qualidade (como Codecov ou SonarCloud), novas funcionalidades complexas ou refatorações profundas em áreas sensíveis do sistema podem ser introduzidas sem nenhuma cobertura de testes associada, sem que os mantenedores ou a comunidade open-source percebam a vulnerabilidade. Consequentemente, o projeto fica suscetível a regressões silenciosas, onde falhas em cenários de exceção não mapeados nas APIs externas e no pipeline de RAG só serão descobertas tardiamente em ambiente de produção.

3.4. Recomendação

Para elevar a maturidade da governança de código e mitigar a invisibilidade métrica detectada na estratégia atual, recomenda-se a implementação das seguintes ações integradas:

1. **Instrumentação e Geração de Métricas:** Alterar a propriedade "test" dentro do bloco "scripts" do package.json raiz para incluir a flag de cobertura, redefinindo o comando para "test": "jest --coverage". Esta pequena modificação

força o motor do Jest a mapear a árvore de execução e exportar relatórios detalhados nos formatos estruturados padrão de mercado.

2. **Visualização Centralizada no Pipeline:** Inserir uma etapa subsequente ao passo Run test suites no arquivo `.github/workflows/run-tests.yaml` para realizar o upload automático dos arquivos de cobertura gerados (localizados em `coverage/lcov.info`) para uma plataforma de análise terceirizada como o Codecov, aproveitando a gratuidade da ferramenta para repositórios públicos.
3. **Implementação de Quality Gates Ativos:** Configurar a chave `coverageThreshold` no arquivo de parametrização global do Jest, estipulando uma meta mínima aceitável de cobertura. Essa barreira atuará como um critério de aceitação automatizado, fazendo com que o GitHub Actions rejeite e bloqueie o merge de qualquer Pull Request que diminua o índice de segurança estipulado para a base de código do sistema.

4. QUALIDADE TÉCNICA DOS TESTES

4.1. Visão geral da suíte de testes

Antes de analisar os critérios individuais, é necessário situar o contexto da suíte de testes do AnythingLLM, pois ele define o pano de fundo de toda a avaliação.

O repositório possui o framework Jest configurado no package.json raiz com o script "test": "jest". Os testes estão organizados dentro da pasta `server/__tests__`, seguindo a estrutura do código de produção em `server/utils`. A única pasta de testes encontrada e documentada pelo DeepWiki é `server/__tests__/utils/vectorDbProviders/pgvector`, contendo um único arquivo: `index.test.js` com 78 linhas e 9 casos de teste.

Esse dado por si só é revelador: em um projeto com mais de 1.764 commits, 195 contribuidores, 40 provedores de LLM, 10 provedores de banco de dados vetorial, pipeline completo de RAG, sistema de agentes, módulo MCP e chat em streaming, a suíte de testes formalmente identificada no repositório cobre uma única função utilitária de sanitização de dados. O próprio CONTRIBUTING.md declara como regra: "Unit test all bug fixes and new features. Your code will not be merged if it doesn't have tests." A análise dos PRs em especial o PR #3077 (Agent Builder, 42 commits, mergeado sem issue vinculada) e o PR #5071 (Native Tool Calling, auto-mergeado em 12 horas com checklist incompleto) demonstra que essa regra não é aplicada de forma consistente.

O diagnóstico geral, portanto, é de uma suíte de testes em estágio inicial de formalização: existe infraestrutura, existe política declarada, e existe pelo menos um arquivo de teste bem construído, mas a cobertura é extremamente localizada em relação à complexidade do sistema.

4.2. Evidências

4.2.1. Comportamento de testes

Evidência	analisada
<code>server/__tests__/utils/vectorDbProviders/pgvector/index.test.js</code> :	
https://github.com/Mintplex-Labs/anything-llm/blob/1cea4df8/server/__tests__/utils/vectorDbProviders/pgvector/index.test.js	

O arquivo contém um bloco “describe” nomeado "PGVector.sanitizeForJsonb" e nove casos com os seguintes nomes:

"returns null/undefined as-is"

"keeps safe whitespace (tab, LF, CR) and removes disallowed C0 controls"

"removes only disallowed control chars; keeps normal printable chars"

"deeply sanitizes objects"

"deeply sanitizes arrays"

"converts Date to ISO string"

"returns primitives unchanged (number, boolean, bigint)"

"returns symbol unchanged"

"does not mutate original objects/arrays"

A nomenclatura adota um padrão consistente e descritivo. Cada nome de teste descreve uma categoria de comportamento observável não um detalhe de implementação. Por exemplo, o teste "does not mutate original objects/arrays" não descreve "chama JSON.parse(JSON.stringify())" (detalhe interno), mas sim o efeito externo que importa para quem usa a função: a garantia de imutabilidade do input. Da mesma forma, "keeps safe whitespace (tab, LF, CR) and removes disallowed C0 controls", especifica precisamente quais caracteres são preservados e quais são removidos comportamento que qualquer desenvolvedor integrador precisa entender.

O bloco “describe” também nomeia corretamente o alvo: PGVector.sanitizeForJsonb, tornando imediato identificar qual unidade está sendo testada. Essa combinação de describe + it segue a convenção de leitura fluente do Jest, onde a frase completa seria: "PGVector.sanitizeForJsonb keeps safe whitespace (tab, LF, CR) and removes disallowed C0 controls" uma sentença que descreve comportamento, não implementação.

Avaliação: os nomes são claros, descritivos e orientados ao comportamento observável. Este é o aspecto mais bem executado da suíte analisada.

4.2.2. Tipos de testes

Evidência analisada: estrutura do arquivo index.test.js e ausência de setup/teardown compartilhado.

https://github.com/Mintplex-Labs/anything-llm/blob/1cea4df8/server/_tests_/utils/vectorDbProviders/pgvector/index.test.js

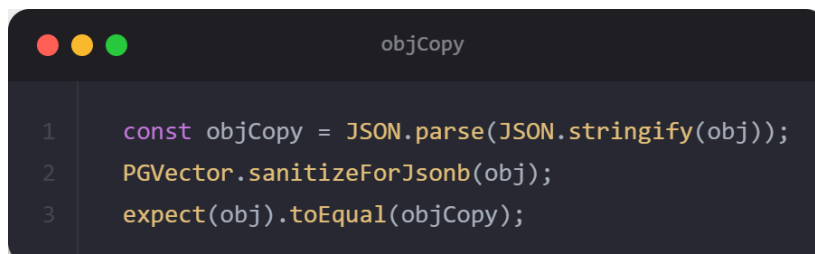
A instância testada é criada uma única vez no escopo do arquivo:



```
index.test.js
1  const { PGVector: PGVectorClass } = require("../utils/vectorDbProviders/pgvector");
2
3  const PGVector = new PGVectorClass();
```

E todos os casos de teste operam sobre essa instância compartilhada. Isso poderia ser um problema se `sanitizeForJsonb` alterasse o estado interno do objeto mas a função é puramente utilitária, sem efeitos colaterais sobre a instância. Cada it recebe seu próprio input literal, executa a função e verifica o resultado de forma completamente isolada dos demais. Não há `beforeEach`, `afterEach`, `beforeAll` ou `afterAll` o que, para funções puras como esta, é adequado: não há estado a preparar nem a limpar.

O teste "does not mutate original objects/arrays" merece atenção especial nesse critério. Ele explicita um requisito de imutabilidade verificando que os objetos originais permanecem inalterados após a chamada:



```
objCopy
1  const objCopy = JSON.parse(JSON.stringify(obj));
2  PGVector.sanitizeForJsonb(obj);
3  expect(obj).toEqual(objCopy);
```

Isso demonstra que o autor teve consciência da necessidade de testar comportamento que poderia introduzir acoplamento entre testes caso o estado fosse compartilhado.

A reprodutibilidade é garantida pela natureza determinística dos inputs: strings literais com caracteres Unicode específicos, objetos e arrays com valores fixos. Não há dependência de relógio, randomização, rede ou banco de dados, o que significa que o teste produz o mesmo resultado independentemente de quando ou onde é executado.

Avaliação: os testes são isolados e reproduzíveis. A única observação é que a instância compartilhada seria um risco em contextos com funções com estado, mas neste caso específico não apresenta problema.

4.3. Diagnóstico

4.3.1. Uso adequado de mocks, stubs, fixtures ou dados de teste

Evidência analisada: estrutura dos dados de teste em index.test.js e ausência de mocks externos.

https://github.com/Mintplex-Labs/anything-llm/blob/1cea4df8/server/_tests_/utils/vectorDbProviders/pgvector/index.test.js

A suíte utiliza dados de teste literais e bem construídos, sem necessidade de mocks ou stubs externos, o que é apropriado para o escopo testado. A função `sanitizeForJsonb` é uma utilidade pura de sanitização de strings e estruturas de dados, sem dependências de banco de dados, rede ou provedores externos. Portanto, a ausência de mocks não é uma lacuna: é uma decisão correta de design de teste.

Os dados de teste são expressivos e cobrem casos limítrofes relevantes. O teste de sanitização de strings inclui caracteres Unicode de controle explicitamente nomeados `\u0000`, `\u0001`, `\u0007`, `\u0008`, `\u001F` tornando visível exatamente quais casos extremos estão sendo verificados. O teste de estruturas aninhadas constrói um objeto com três níveis de profundidade para verificar a recursividade da função:



```
teste de estruturas aninhadas
1  const input = {
2    plain: "ok",
3    bad: "has\u0000nul",
4    nested: {
5      arr: ["fine", "bad\u0001", { deep: "\u0002oops" }],
6    },
7  };

```

Isso constitui uma fixture bem construída: dados de teste que são ao mesmo tempo mínimos (sem informação irrelevante) e suficientes para exercitar o comportamento esperado.

A ausência de mocks é, contudo, um indicador importante quando considerada em relação ao restante do sistema. Os componentes de maior risco do AnythingLLM o pipeline de RAG, os providers de LLM, o sistema de agentes, o módulo MCP dependem intensamente de APIs externas. Para testá-los de forma isolada, seria necessário o uso de mocks e stubs que simulem as respostas dessas APIs. A ausência de testes nessas camadas, confirmada pela estrutura do repositório, significa que o padrão de mocking necessário para cobertura adequada simplesmente não foi estabelecido no projeto. O único caso de teste existente trata de uma função que, por ser pura, não demanda esse padrão, o que torna difícil avaliar como o time abordaria mocking em contextos mais complexos.

Avaliação: dentro do escopo testado, os dados de teste são adequados, expressivos e cobrem casos limítrofes relevantes. A ausência de mocks é justificada pelo tipo de função testada, mas aponta para uma lacuna estrutural: não há evidência de como o sistema lidaria com testes de componentes que dependem de serviços externos.

4.4. Risco

A classificação de risco está definida como médio-alto, mas não deriva de uma falha no teste analisado — que, como visto, é tecnicamente correto — mas da assimetria estrutural entre cobertura existente e superfície de risco do sistema. Para justificar essa classificação de forma rigorosa, é necessário responder primeiro a duas perguntas centrais da auditoria, pois elas são o fundamento técnico do risco.

4.4.1. Validação de regras importantes vs. detalhes superficiais

O teste em `index.test.js` valida uma regra real de comportamento, não um detalhe de implementação: a função `sanitizeForJsonb` precisa garantir que dados armazenados em colunas JSONB do PostgreSQL não contenham caracteres de controle inválidos, que poderiam corromper o armazenamento ou causar falhas de parsing no banco. Os nove casos cobrem essa regra de ponta a ponta — entradas nulas, whitespace seguro, caracteres de controle proibidos, estruturas aninhadas, tipos primitivos e a garantia de imutabilidade. Isso é, em si, uma regra de negócio relevante para a camada de persistência vetorial, não um teste artificial criado apenas para gerar métrica de cobertura.

O problema surge quando se eleva o nível de análise do arquivo individual para o sistema como um todo. As regras de negócio que realmente definem o comportamento crítico do AnythingLLM — e que, portanto, deveriam ser o foco prioritário de qualquer suíte de testes madura — são outras:

Roteamento correto entre os 40 provedores de LLM suportados: nenhuma evidência de teste automatizado garantindo que a troca de provedor (OpenAI, Anthropic, Ollama, etc.) preserve o contrato de resposta esperado pelo restante do sistema.

Integridade do pipeline de RAG: a etapa de chunking, embedding e recuperação de contexto — o núcleo funcional do produto — não possui testes que validem se o contexto recuperado é relevante ou se o pipeline se comporta corretamente sob falhas parciais (ex: timeout de um provedor de embedding).

Comportamento do sistema de agentes: o PR #3077 (Agent Builder), com 42 commits e mesclado sem issue vinculada, introduziu uma funcionalidade inteira sem rastro de teste correspondente na estrutura do repositório.

Segurança e parsing do módulo MCP: dado que MCP envolve execução de ferramentas externas, a ausência de testes aqui é particularmente sensível, pois erros de parsing ou validação de payload têm potencial de impacto em segurança, não apenas em funcionalidade.

Portanto, a resposta direta é: a maioria dos testes existentes valida uma regra real, mas de importância secundária. As regras que sustentam a proposta de valor central do produto — multiprovedor, RAG, agentes, MCP — não são validadas por nada. Isso significa que a suíte de testes, tomada como um todo, não protege o sistema contra os tipos de regressão que mais importariam para o usuário final ou para a operação do produto.

4.4.2. Testes redundantes, frágeis ou difíceis de manter

Dentro do arquivo `index.test.js`, não há evidência de redundância, fragilidade ou dificuldade de manutenção. Cada um dos nove casos it cobre uma categoria de input mutuamente exclusiva (null/undefined, whitespace seguro, controles proibidos, objetos aninhados, arrays, datas, primitivos, símbolos, imutabilidade), sem sobreposição de propósito. Os dados são literais e determinísticos — não há geração aleatória, dependência de relógio, rede ou ordem de execução entre testes —, o que

os torna estáveis frente a refatorações que não alterem o contrato da função. Não há `beforeEach/afterEach` acoplando estado entre casos, o que elimina uma fonte comum de fragilidade em suítes Jest mal construídas (testes que passam ou falham dependendo da ordem de execução).

Entretanto, ao reformular a pergunta em um nível de sistema, surge um tipo distinto de fragilidade — não fragilidade *de teste*, mas fragilidade estrutural por ausência de teste: a manutenibilidade de longo prazo do projeto está comprometida não porque os testes existentes quebram sem motivo, mas porque a vasta maioria do código de produção não possui rede de proteção alguma. Em termos práticos, isso significa que qualquer refatoração nos módulos de maior complexidade ciclomática — já identificados na análise SOLID como God Objects (`systemSettings.js`, `updateENV.js`) — é, hoje, um ato de fé: não há teste que confirme ou refute se o comportamento foi preservado. Um teste frágil pelo menos avisa (mesmo que com falso positivo); a ausência total de teste não avisa nada, e esse silêncio é a forma mais perigosa de "dificuldade de manutenção", porque ela não aparece como erro — aparece como ausência de sinal.

4.4.3. Síntese da justificativa de risco

Combinando as duas respostas, o risco é classificado como MÉDIO-ALTO pelos seguintes vetores concretos:

Cobertura desproporcional à criticidade: 1 arquivo de teste, 9 casos, para uma função utilitária, frente a 40 provedores de LLM, 10 bancos vetoriais, RAG, agentes e MCP sem cobertura.

Política declarada e não aplicada: o `CONTRIBUTING.md` exige testes para todo bug fix e feature ("Your code will not be merged if it doesn't have tests"), mas os PRs #3077 e #5071 (auto-mergeado em 12h com checklist incompleto) demonstram que a regra é sistematicamente contornada.

Ausência de precedente para mocking de dependências externas: como discutido na seção de Diagnóstico anterior, o único teste trata de função pura; não há padrão estabelecido de como a equipe simularia respostas de LLM ou bancos vetoriais para testar os componentes que mais precisam disso.

Refatoração sem rede de segurança: os módulos de maior acoplamento (God Objects) são também os que mais mudam e os que têm menos proteção, uma combinação que maximiza a probabilidade de regressão não detectada.

4.5. Recomendação

As recomendações estão organizadas por ordem de impacto/esforço, partindo das ações que fecham as lacunas mais críticas identificadas.

Priorização de cobertura por risco de negócio, não por linhas de código: Iniciar a expansão da suíte pelos pontos identificados como estruturalmente críticos na análise SOLID: os 9 arquivos que precisam ser tocados para adicionar um novo provedor de LLM (o inverso do "Swap Test"), e os God Objects `systemSettings.js` e `updateENV.js`. Esses são os pontos onde uma regressão silenciosa teria o maior raio de impacto, exatamente porque concentram múltiplas responsabilidades em um único arquivo.

Estabelecer um padrão de mocking para dependências externas: Criar uma camada reutilizável de fixtures e mocks para simular respostas de provedores de LLM e bancos vetoriais. O estilo de dados literais já usado em `index.test.js` é um bom ponto de partida estilístico — a equipe já demonstrou capacidade de escrever fixtures mínimas e suficientes; falta apenas estender esse cuidado para casos com I/O simulado (ex: usar `jest.mock()` para interceptar chamadas HTTP a provedores de LLM, retornando respostas determinísticas).

Testes de contrato para o pipeline de RAG e roteamento de providers: Em vez de tentar testar exaustivamente cada um dos 40 provedores, recomenda-se um teste de contrato (contract test): definir a interface mínima que todo provedor deve cumprir e testar essa interface contra um provedor real (em ambiente de CI com chave de teste) e contra mocks. Isso valida a regra de negócio real — "qualquer provedor suportado deve responder de forma compatível" — em vez de detalhes de implementação de cada integração.

Gate de CI vinculado à política do CONTRIBUTING.md: Configurar o pipeline de CI para bloquear merge de PRs que alterem arquivos em `server/utils/` ou `server/models/` sem testes correspondentes adicionados ou atualizados. Isso fecha a brecha concreta demonstrada pelo PR #5071 (auto-merge com checklist incompleto), transformando a política de "regra declarada" em "regra tecnicamente verificável".

Meta de cobertura incremental ("boy scout rule"), não retroativa: Não é realista exigir cobertura total do código legado de uma vez, dado o volume do projeto (1.764+ commits, 195 contribuidores). A recomendação prática é adotar a regra de que todo PR que modifica um arquivo sem teste deve adicionar cobertura mínima para a função especificamente alterada — isso cria uma trajetória de melhoria contínua sem bloquear o ritmo de desenvolvimento.

Auditoria periódica de PRs de alto risco: Dado que o PR #3077 (Agent Builder) foi mergeado sem issue vinculada e com 42 commits, recomenda-se instituir uma revisão obrigatória de segundo nível (sign-off de um mantenedor sênior) para PRs que introduzem funcionalidade nova de grande porte, especificamente verificando a presença de testes antes da aprovação — fechando, na prática de revisão humana, o que o gate de CI fecha tecnicamente.

5. LACUNAS, RISCOS E FALHAS POTENCIAIS

5.1. Evidências

5.1.1. Funcionalidades críticas sem testes

O mapeamento arquitetural e de modelagem realizado na Atividade 1 identificou que o sistema AnythingLLM opera fundamentalmente em torno de dois fluxos críticos: a ingestão de dados (via módulo Collector) e a orquestração da geração de respostas (Chat via pipeline RAG). Toda essa engrenagem é sustentada pelo conceito estrutural de *workspaces*, que atuam como silos dedicados para organizar os documentos enviados pelos usuários, os *embeddings* armazenados no banco de dados vetorial e as configurações específicas de limite e contexto de cada ambiente.

Contudo, a inspeção minuciosa do diretório `server/__tests__` (e seus subdiretórios `/models` e `/utils`) revela que a suíte de testes automatizados atual foca estritamente em funções utilitárias e regras de negócio isoladas — como a quebra de textos (TextSplitter) e a sanitização de strings. Não foi encontrado no repositório nenhum arquivo de teste de integração, de sistema ou *end-to-end* (E2E) que valide a criação de *workspaces*, o isolamento dos vetores durante a ingestão ou a correta segregação de contexto na hora em que o servidor monta o *prompt* para consultar a LLM externa.

Além disso, o fluxo de comunicação de *chat* (como o envio de mensagens em *streaming* e a recuperação de contexto vetorial) possui alta complexidade lógica no código-fonte, mas também opera sem a cobertura de testes que simulem a interação do usuário e garantam que as respostas geradas utilizem apenas os documentos pertencentes ao *workspace* ativo.

```
const { contextTexts, sourceDocuments } = await VectorDb.performSimilaritySearch({
  namespace: workspace.slug,
  input: message,
  LLMConnector,
  similarityThreshold: workspace?.similarityThreshold,
  topN: workspace?.topN,
});

// ... (algumas linhas de logs e tratamento de embeddings falsos) ...

const LLMConnector = getLLMProvider({
  provider: workspace?.chatProvider,
  model: workspace?.chatModel,
});

const systemPrompt = await chatPrompt(workspace);
const messages = await LLMConnector.compressMessages(
  {
    systemPrompt,
    userPrompt: message,
    contextTexts,
    chatHistory,
  },
  process.env.OPENAI_KEY
);
```

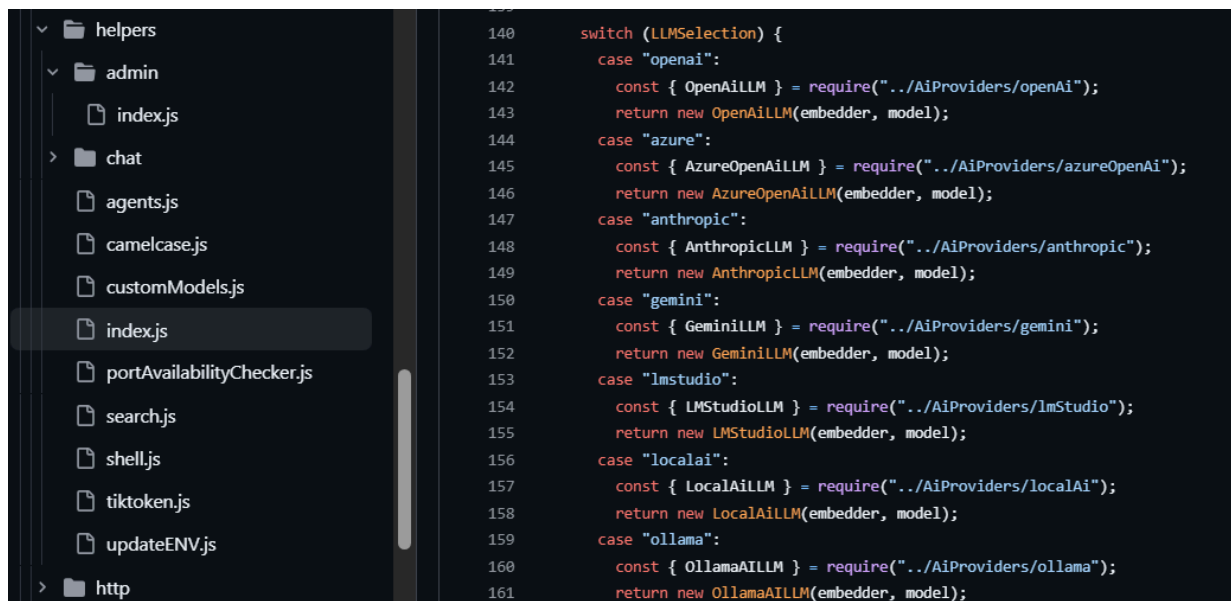
</server/utils/chats/stream.js>

5.1.2. Módulos que apresentam riscos para alteração

O mapeamento arquitetural conduzido na Atividade 2 diagnosticou um acúmulo significativo de dívida técnica em dois componentes centrais do servidor: a Fachada de gerenciamento de arquivos (`server/utils/files/index.js`), que assumiu responsabilidades excessivas de validação e *parsing* (atuando como um *God Object*), e a lógica de instanciação dos provedores de IA, especificamente a função fábrica `getLLMProvider`, classificada como um *God Switch* devido à sua estrutura procedural massiva e violação do princípio Aberto/Fechado (*Open/Closed Principle*).

Ao cruzar esse diagnóstico arquitetural com a auditoria de testes atual, constata-se uma completa ausência de cobertura automatizada (testes unitários ou de integração) focada nesses módulos críticos. A varredura no diretório `server/__tests__` e em suas subpastas confirma que o repositório não possui arquivos de teste dedicados a validar o comportamento esperado do `getLLMProvider` ou as lógicas complexas de metadados da Fachada de arquivos.

switch/case sem testes



```
140 switch (LLMSelection) {
141   case "openai":
142     const { OpenAiLLM } = require("../AiProviders/openAi");
143     return new OpenAiLLM(embedder, model);
144   case "azure":
145     const { AzureOpenAiLLM } = require("../AiProviders/azureOpenAi");
146     return new AzureOpenAiLLM(embedder, model);
147   case "anthropic":
148     const { AnthropicLLM } = require("../AiProviders/anthropic");
149     return new AnthropicLLM(embedder, model);
150   case "gemini":
151     const { GeminiLLM } = require("../AiProviders/gemini");
152     return new GeminiLLM(embedder, model);
153   case "lmstudio":
154     const { LMStudioLLM } = require("../AiProviders/lmStudio");
155     return new LMStudioLLM(embedder, model);
156   case "localai":
157     const { LocalAiLLM } = require("../AiProviders/localAi");
158     return new LocalAiLLM(embedder, model);
159   case "ollama":
160     const { OllamaAiLLM } = require("../AiProviders/ollama");
161     return new OllamaAiLLM(embedder, model);
```

</server/utils/helpers/index.js>

A inexistência de uma suíte de testes cobrindo essas lógicas altamente acopladas significa que o comportamento atual dessas classes não está mapeado de forma executável. Como a Atividade 2 recomendou refatorações profundas nestes exatos arquivos (como a introdução do padrão *Registry* para as IAs e a quebra da Fachada em classes especialistas menores), a falta dessa rede de testes de segurança automatizada é uma evidência clara de que qualquer alteração nesses módulos no estado atual caracteriza uma operação "no escuro", com altíssima probabilidade de gerar regressões não detectadas no sistema.

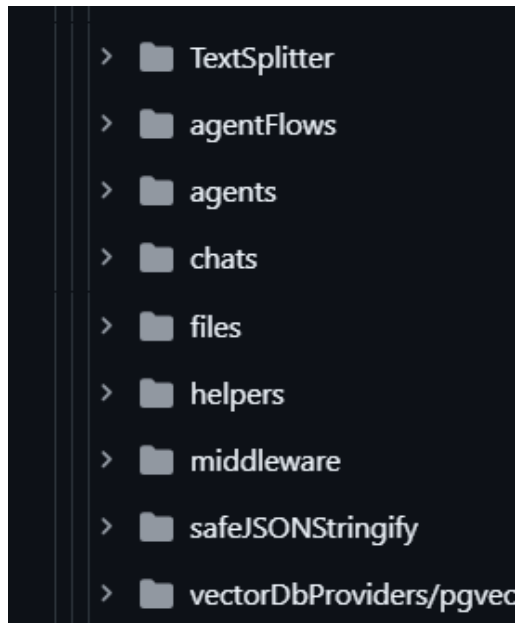
5.1.3. Dependências externas

O *core business* do AnythingLLM depende fundamentalmente da integração com serviços de terceiros, orquestrando dezenas de provedores de Modelos de Linguagem (LLMs) e bancos de dados vetoriais em nuvem. Embora a suíte de testes utilize a função `jest.mock` para isolar e simular com sucesso o banco de dados relacional interno (Prisma), a auditoria revelou que o projeto não possui uma estratégia técnica implementada para isolar a comunicação HTTP com as APIs externas de IA.

A inspeção detalhada do diretório `server/__tests__/utils/vectorDbProviders/` confirmou a existência de apenas um arquivo de teste (`pgvector/index.test.js`). A análise deste arquivo demonstra que a suíte valida exclusivamente uma função utilitária pura de sanitização de *strings* (`sanitizeForJsonb`), sem realizar qualquer simulação de conexão de rede ou operação real de leitura e escrita vetorial.

Adicionalmente, ao buscar por testes nas pastas responsáveis pelos provedores de Inteligência Artificial (como `server/utils/AiProviders/`), constatou-se a total ausência de arquivos `.test.js`. O repositório não utiliza bibliotecas de interceptação de rede (como o `nock` ou recursos avançados do próprio Jest) para simular o comportamento de serviços como OpenAI, Anthropic ou Pinecone. Não existem *stubs* ou *fixtures* (arquivos JSON com exemplos de respostas falsas das APIs) que permitam à aplicação testar como ela se comportaria ao receber um *payload* da nuvem.

Ausência de AiProviders na pasta de testes



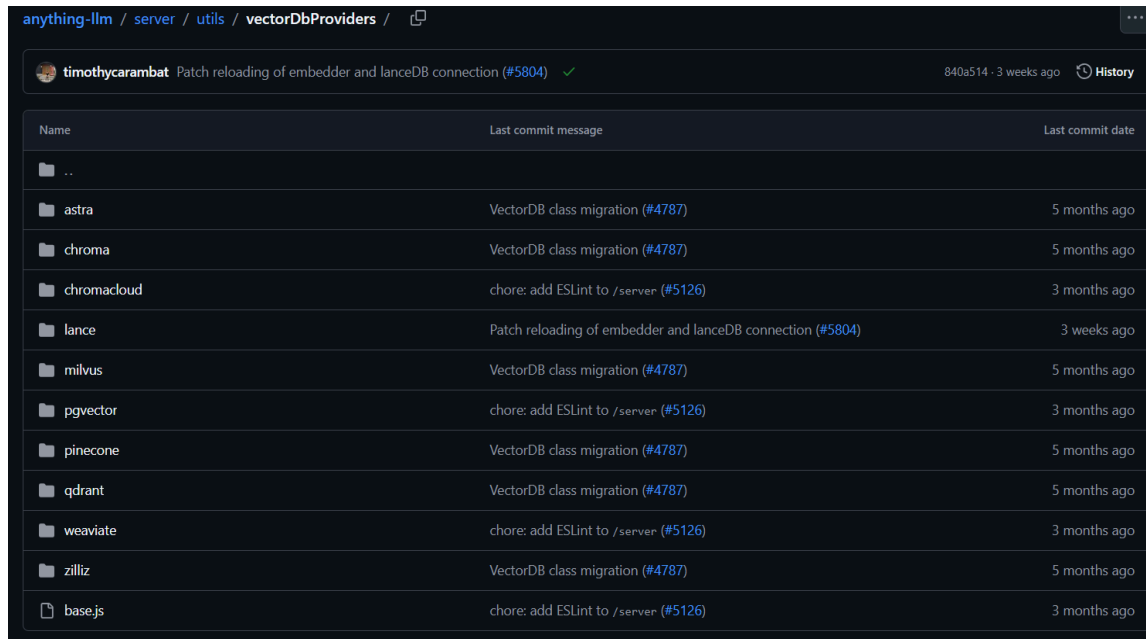
[server/](#) [tests](#) [/utils](#)

Provedores de LLM: Nenhum dos mais de 30 provedores de LLM registrados dentro de `getLLMProvider` possui teste unitário, de contrato, ou de integração dedicado. Cada case do switch instancia uma classe de provider com lógica própria de autenticação, formatação de mensagens e tratamento de streaming, e nenhuma dessas classes é exercitada por uma suíte automatizada, nem mesmo com mocks simulando as respostas das respectivas APIs.

Bancos de dados vetoriais: Como já evidenciado na seção 4 deste relatório, a única cobertura de teste relacionada a bancos vetoriais é o arquivo `index.test.js` dentro da pasta `pgvector` e mesmo essa cobertura está restrita a uma única função utilitária, `sanitizeForJsonb`, responsável por sanitizar dados antes da inserção. Os outros oito provedores, incluindo o LanceDB (que é o banco vetorial padrão da aplicação, usado por toda instalação que não configura explicitamente outro), não têm nenhum arquivo de teste correspondente dentro de `server/__tests__/utils/vectorDbProviders/`. A imagem a seguir compara, lado a lado, a lista de providers efetivamente implementados no código de produção com a lista de providers que possuem alguma pasta de teste equivalente.

O contraste chama atenção: de nove implementações de banco vetorial mantidas no código de produção, apenas uma, `pgvector`, tem uma pasta de teste

dedicada, e ainda assim cobrindo só uma função de sanitização, não a conexão real, a execução de queries vetoriais ou o tratamento de falhas de rede.



The screenshot shows a GitHub repository page for 'anything-llm' at the path 'server / utils / vectorDbProviders'. It displays a commit history table with columns for Name, Last commit message, and Last commit date. The table lists various vector database providers and their corresponding commit messages and dates.

Name	Last commit message	Last commit date
..		
astra	VectorDB class migration (#4787)	5 months ago
chroma	VectorDB class migration (#4787)	5 months ago
chromacloud	chore: add ESLint to /server (#5126)	3 months ago
lance	Patch reloading of embedder and lanceDB connection (#5804)	3 weeks ago
milvus	VectorDB class migration (#4787)	5 months ago
pgvector	chore: add ESLint to /server (#5126)	3 months ago
pinecone	VectorDB class migration (#4787)	5 months ago
qdrant	VectorDB class migration (#4787)	5 months ago
weaviate	chore: add ESLint to /server (#5126)	3 months ago
zilliz	VectorDB class migration (#4787)	5 months ago
base.js	chore: add ESLint to /server (#5126)	3 months ago

<https://github.com/Mintplex-Labs/anything-llm/tree/master/server/utils/vectorDbProviders>

5.1.4. Proteção contra regressões em fluxos centrais

É possível observar a inexistência de uma malha de testes *End-to-End* (E2E) ou de testes de regressão sistêmicos automatizados para os fluxos críticos da aplicação. Uma inspeção aprofundada nos arquivos de orquestração do projeto revela um foco exclusivo na camada unitária, deixando a integração real do sistema completamente descoberta.

Primeiro, a análise declarativa do arquivo `package.json` na raiz do projeto demonstra que a única ferramenta de teste configurada globalmente é o Jest, invocada de forma simples através do script `"test": "jest"`. Uma varredura no bloco `devDependencies` desse arquivo, bem como nos manifestos dos submódulos (`server/package.json` e `frontend/package.json`), confirma a ausência de bibliotecas ou *frameworks* padrão de mercado para testes de interface e fluxos contínuos, como Cypress, Playwright, Selenium ou Puppeteer voltado à navegação de UI. Não há nenhum script do tipo `"test:e2e"` configurado para simular o comportamento do usuário final.

```

"scripts": {
  "test": "jest",
  "lint": "cd server && yarn lint && cd ../frontend && yarn lint && cd ../collector && yarn lint",
  "lint:ci": "cd server && yarn lint:check && cd ../frontend && yarn lint:check && cd ../collector && yarn lint:check",
  "setup": "cd server && yarn && cd ../collector && yarn && cd ../frontend && yarn && cd .. && yarn setup:envs && yarn prisma:setup && echo \"Please run yarn dev\"",
  "setup:envs": "cp -n ../frontend/.env.example ../frontend/.env; cp -n ../server/.env.example ../server/.env.development; cp -n ../collector/.env.example ../collector/.env.development",
  "dev:server": "cd server && yarn dev",
  "dev:collector": "cd collector && yarn dev",
  "dev:frontend": "cd frontend && yarn dev",
  "dev": "npm concurrently \"yarn dev:server\" \"yarn dev:frontend\" \"yarn dev:collector\"",
  "prisma:generate": "cd server && npx prisma generate",
  "prisma:migrate": "cd server && npx prisma migrate dev --name init",
  "prisma:seed": "cd server && npx prisma db seed",
  "prisma:setup": "yarn prisma:generate && yarn prisma:migrate && yarn prisma:seed",
  "prisma:reset": "truncate -s 0 server/storage/anythingllm.db && yarn prisma:migrate",
  "prod:server": "cd server && yarn start",
  "prod:frontend": "cd frontend && yarn build",
  "generate:cloudformation": "node cloud-deployments/aws/cloudformation/generate.mjs",
  "generate:gcp_deployment": "node cloud-deployments/gcp/deployment/generate.mjs",
  "translations:verify": "cd frontend/src/locales && node verifyTranslations.mjs",
  "translations:normalize": "cd frontend/src/locales && node normalizeEn.mjs && cd ../../ && cd frontend && yarn lint && cd .. && yarn translations:verify",
  "translations:prune": "cd frontend/src/locales && node findUnusedTranslations.mjs --delete && cd ../../ && yarn translations:normalize",
  "translations:create": "cd extras/translator && node index.mjs --all && cd ../../frontend && yarn lint && cd ../../",
},
"private": false,
"devDependencies": {
  "concurrently": "^9.1.2",
  "jest": "^29.7.0"
}

```

[package.json](#)

Segundo, o rastreamento da esteira de Integração Contínua (CI/CD) gerida pelo GitHub Actions atesta categoricamente essa limitação de escopo. O arquivo de *workflow* `.github/workflows/run-tests.yaml` está rigidamente programado para preparar o ambiente instalando dependências, acionar o `yarn prisma:setup` para as migrações do banco de dados relacional e, em seguida, disparar globalmente o comando unificado `yarn test`. Não existe nenhum passo (*step*) ou tarefa (*job*) no pipeline que suba simultaneamente o servidor Node.js e o *frontend* em React/Vite para realizar uma validação sistêmica de rotas críticas — como a criação ponta a ponta de um *workspace* seguida de uma interação no chat.

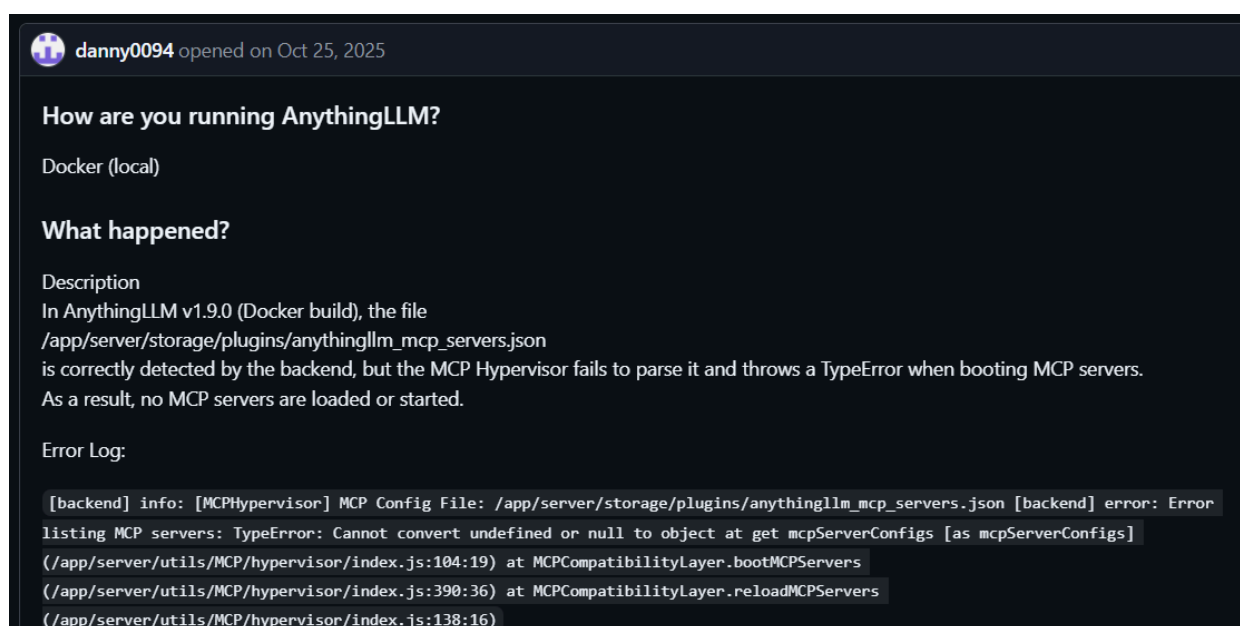
Por fim, a exploração física da estrutura de diretórios do repositório corrobora os achados estáticos. Não existem pastas convencionalmente dedicadas à automação de jornadas completas (como `cypress/e2e/`, `tests/e2e/` ou `integration/`). Todo o escopo de validação documentado encontra-se restrito à pasta `server/__tests__`, cujos arquivos limitam-se a validar módulos e funções de utilidade pura, sem testar o acoplamento de rede (chamadas HTTP e WebSockets para o *streaming* de respostas) que conecta as ações da interface gráfica ao motor orquestrador do *backend*.

5.1.5. Eficácia da suíte na detecção rápida de bugs severos

O módulo MCP é a integração mais recente do sistema e, arquiteturalmente, também a mais arriscada, porque delega execução a processos externos arbitrários, servidores MCP conectados via stdio, SSE ou HTTP streaming, configurados livremente pelo usuário final. Uma busca pelo termo MCP dentro de `server/__tests__` não retorna nenhum resultado, confirmando ausência total de cobertura.

Essa lacuna não é apenas uma preocupação teórica de auditoria: o próprio histórico de Issues públicas do repositório documenta falhas reais nessa camada. Uma delas registra um erro de runtime — `TypeError: Cannot convert undefined or null to object`, disparado no getter `mcpServerConfigs`, que impede o boot de todos os servidores MCP configurados em ambientes Docker. Outra, um Pull Request (#4299), foi aberta especificamente para corrigir condições de corrida na criação de instâncias do `MCPHypervisor`, vazamentos de memória em padrões `Promise.race` com `timeout`, e ausência de tratamento de erro em operações assíncronas como `ping()` e `listTools()`. São, portanto, falhas que já aconteceram em produção, num módulo que continua sem nenhuma linha de teste automatizado até o momento desta auditoria.

A imagem seguinte reproduz o log de erro real documentado na issue #4579([BUG]: MCP servers fail to load from `anythingllm_mcp_servers.json` in Docker (`TypeError: Cannot convert undefined or null to object`)), ao lado da confirmação de que a busca por testes relacionados ao módulo retorna vazio.



<https://github.com/Mintplex-Labs/anything-llm/issues/4579>

A convergência dessas três evidências, provedores de LLM, bancos vetoriais e módulo MCP, mostra um padrão consistente: a estratégia de testes do AnythingLLM concentra-se quase inteiramente em funções utilitárias puras e deixa descoberta justamente a camada que sustenta a proposta de valor central do produto. É também, não por coincidência, a camada mais difícil de testar — porque depende de contratos de terceiros, disponibilidade de rede e, no caso do MCP, processos externos fora do controle direto do projeto. Mas dificuldade de testar não é o mesmo que dispensa de testar; é, se algo, um argumento a favor de priorizar exatamente essa camada primeiro.

5.2. Diagnóstico

Olhando as três evidências da seção 5.1 em conjunto, e não isoladamente, fica visível um padrão único, que atravessa fluxos de negócio, dívida técnica arquitetural e dependências externas: a suíte de testes do AnythingLLM protege bem um núcleo estreito de funções utilitárias puras e deixa descoberto justamente aquilo que é mais complexo, mais acoplado e mais sujeito a falhas reais.

Convém detalhar essa convergência em três camadas, porque cada uma carrega um tipo de risco diferente.

5.2.1. Camada de fluxo crítico de negócio

Os pilares funcionais do produto, ingestão de documentos pelo Collector, geração de embeddings, isolamento de contexto por workspace e orquestração do pipeline de RAG durante o streaming do chat, não têm nenhum teste de integração ou end-to-end. A suíte atual testa partes isoladas, como o TextSplitter ou a sanitização de strings, mas nunca o fluxo completo que um usuário de fato percorre quando faz upload de um documento e depois conversa com ele. É a diferença entre testar se uma peça de motor funciona sozinha e testar se o carro liga.

5.2.2. Camada de dívida técnica arquitetural

Os dois anti-padrões identificados nas Atividades anteriores, o God Object em `server/utils/files/index.js` e o God Switch em `getLLMProvider`, concentram lógica condicional extensa e, justamente por isso, alta probabilidade de erro humano durante manutenção. E são, ao mesmo tempo, os módulos com zero cobertura de teste documentada. Refatorar código sem testes não é impossível, mas é arriscado:

qualquer alteração nessas funções vira uma aposta sobre se o comportamento permanece o mesmo, sem nenhuma forma automatizada de confirmar isso antes do deploy.

5.2.3. Camada de dependências externas

A multiplicidade de integrações que sustenta a proposta de valor do AnythingLLM, "conecte qualquer modelo, qualquer banco vetorial, qualquer ferramenta via MCP" é, paradoxalmente, a camada menos testada do sistema. A cobertura real, hoje, está limitada a uma única função de sanitização para um único provedor de banco vetorial. Os mais de 30 provedores de LLM e o módulo MCP completo seguem sem qualquer verificação automatizada, mesmo havendo, no caso do MCP, histórico documentado de falhas reais em produção.

O padrão geral que emerge dessas três camadas é o de uma suíte de testes invertida em relação ao que normalmente se recomenda. O ideal seria uma pirâmide: uma base ampla de testes unitários rápidos, sustentando uma camada intermediária de testes de integração, coroada por um número menor de testes end-to-end. No AnythingLLM, a base de testes unitários existe e é de boa qualidade, como discutido na seção 4, mas a camada intermediária de integração simplesmente não foi construída. O resultado é uma pirâmide sem meio: boa fundação, nenhuma estrutura no centro, e o topo (fluxos completos de usuário) nunca verificado de forma automatizada.

5.2.4. Risco de Regressão nos Fluxos de RAG e Ingestão

Vale registrar que esse diagnóstico não é uma crítica à equipe que mantém o projeto, nem implica negligência. AnythingLLM é um projeto open-source de grande porte, com centenas de contribuidores e um ritmo de lançamento de funcionalidades muito acima da média. O que a auditoria revela é simplesmente que a prática de testes não acompanhou, na mesma velocidade, o crescimento da superfície de integrações; um padrão, aliás, bastante comum em projetos que crescem rápido e que tendem a testar primeiro o que é mais simples de testar.

Analisando a fundo, esta falta de proteção contra regressões no fluxo principal acontece por três razões:

1. **Efeitos colaterais invisíveis:** Como as etapas de RAG (upload, vetorização e resposta da IA) estão fortemente ligadas e dependem de chamadas de rede assíncronas, mexer numa ponta do código pode partir a outra sem que ninguém perceba. Sem testes integrados, o pipeline do GitHub Actions fica completamente cego a estes impactos indiretos.
2. **A "fuga" à complexidade dos Mocks:** O ecossistema falha ao não criar um padrão para simular as APIs externas (como OpenAI ou Pinecone). O diagnóstico indica que, para evitar a complexidade de criar cenários simulados (*stubs* ou *fixtures*) para estes serviços de terceiros, a equipa de desenvolvimento optou por deixar a parte mais importante do sistema sem cobertura nenhuma.
3. **Falta de validação no isolamento de dados:** O produto vive da promessa de manter a separação rígida de dados entre diferentes *workspaces*. No entanto, não há um único teste automatizado que valide se uma falha de concorrência ou um vazamento de memória poderia expor os documentos de um ambiente noutro chat.

5.3. Risco

Os três fatores evidenciados na seção 5.1 não atuam de forma isolada; eles se amplificam mutuamente, e vale percorrer esse encadeamento com calma:

5.3.1. Refatoração no escuro.

Como `getLLMProvider` não tem testes, qualquer tentativa de aplicar a refatoração recomendada na Atividade 2, introduzir o padrão Registry para substituir

o switch/case, corre o risco real de quebrar silenciosamente a integração com um ou mais dos 30+ provedores de LLM suportados. Essa quebra só seria percebida manualmente, porque não existe teste de contrato garantindo que cada provider continue respondendo ao formato esperado.

5.3.2. Vazamento de Contexto entre Workspaces

Alterações no pipeline de streaming de chat (server/utils/chats/stream.js) podem comprometer o isolamento entre workspaces sem que nenhum teste capture a regressão. Isso não é um risco puramente funcional: é, antes de tudo, um risco de privacidade e segurança de dados, já que um usuário poderia, em tese, receber uma resposta gerada a partir de documentos pertencentes a outro workspace.

5.3.3. Falhas Silenciosas em integrações externas.

Sem testes de contrato cobrindo os provedores de LLM e bancos vetoriais, mudanças de comportamento em APIs de terceiros, uma alteração de schema, uma nova política de rate limiting, um campo que passa a vir vazio só são percebidas quando já estão afetando usuários reais em produção.

5.3.4. Precedente documento de revisão permissiva.

O projeto é mantido por uma comunidade ativa, com mais de 195 contribuidores únicos, conforme já mencionado na seção 4.1. Isso é uma virtude para a velocidade de desenvolvimento, mas amplia a superfície de risco: o próprio histórico de Pull Requests do repositório já registrou casos de merge com checklist de testes incompleto, o PR #5071 (Native Tool Calling), por exemplo, foi auto-mergeado em 12 horas sem checklist completo, apesar de o CONTRIBUTING.md do projeto declarar explicitamente que código sem testes não deveria ser aceito.

Posto de outra forma: a probabilidade de uma regressão silenciosa nessas áreas críticas não é hipotética. Há precedente documentado de processo de revisão permissivo o suficiente para permitir merges sem a cobertura que o próprio projeto exige de si mesmo por escrito. Isso eleva o risco teórico para concreto, e é exatamente esse salto que justifica a classificação Alto, e não Médio, para esta seção.

5.4. Recomendação

Diante de um risco classificado como Alto, a recomendação central é simples de enunciar e difícil de simplificar mais do que isso: testar primeiro o que é mais arriscado, não o que é mais fácil de testar. As quatro ações abaixo seguem essa lógica, em ordem de prioridade sugerida.

5.4.1. Testes de contrato para integrações externas.

Antes de qualquer refatoração estrutural, implementar testes que validem o contrato de interface esperado por cada provedor de LLM e de banco vetorial, por exemplo, garantir que todo provider implemente corretamente métodos como `compressMessages` e `streamChat`. Esses testes devem usar mocks para simular respostas, timeouts e erros das APIs externas, sem depender de chamadas reais a serviços de terceiros durante a execução da suíte de CI.

5.4.2. Testes de integração para o fluxo de Workspace e RAG.

Criar uma suíte de testes de integração; mesmo que apoiada em banco de dados em memória ou containerizado via Docker no próprio pipeline de CI que valide especificamente o isolamento de contexto entre workspaces durante a busca por similaridade (`performSimilaritySearch`). O objetivo é garantir, de forma automatizada e repetível, que documentos de um workspace nunca sejam recuperados em consultas originadas de outro.

5.4.3. Testes de caracterização antes de refatorar

Antes de aplicar as refatorações já recomendadas na Atividade 2; quebra do God Object em `files/index.js` e introdução do Registry em `getLLMProvider`, escrever testes de caracterização que documentem o comportamento atual dessas funções, mesmo que esse comportamento não seja o ideal. Esse tipo de teste cria uma rede de segurança mínima: ele não valida que o código está certo, valida que o comportamento observável não mudou depois da refatoração, o que já é suficiente para reduzir drasticamente o risco de regressão.

5.4.4. Priorização por risco combinado, não por cobertura percentual.

Em vez de perseguir uma meta genérica de cobertura percentual, como a recomendada na seção 3.4 para o projeto como um todo, priorizar a criação de novos testes pelos módulos de maior risco combinado, calculado como o produto de três fatores: frequência de alteração, grau de acoplamento e dependência de serviços externos. Nesse cálculo, o pipeline de streaming de chat e a função `getLLMProvider` aparecem no topo da lista, e deveriam ser os primeiros a receber atenção, antes de qualquer expansão de cobertura em áreas de menor criticidade.

5.4.5. Implementação de Testes de Regressão Automatizados nos Fluxos de RAG e Ingestão

É urgente que a equipa de desenvolvimento pare de tratar o *AnythingLLM* como um conjunto de funções isoladas e passe a protegê-lo como a plataforma integrada que ele realmente é. A recomendação prática é a criação de uma suíte de testes de integração e testes End-to-End (E2E) que simulem o comportamento real do utilizador. Isto significa automatizar o fluxo completo: criar um *workspace* de teste, simular o upload de um documento fictício através do *Collector*, validar a fragmentação e, por fim, disparar uma pergunta no chat.

5.4.6. Como executar isto na prática

1. **Criar Padrões de Mocking e Fixtures:** Em vez de fazer chamadas reais para as APIs da OpenAI ou Pinecone (o que tornaria os testes caros e instáveis), a equipa deve criar ficheiros JSON estáticos (*fixtures*) que imitem exatamente as respostas destas ferramentas externas. O Jest deve ser configurado para intercetar as chamadas de rede e devolver estes dados simulados.

2. **Validar Barreiras de Isolamento (*Multi-tenancy*):** Desenhar testes específicos onde dois ambientes virtuais temporários sejam criados ao mesmo tempo. O teste deve forçar uma pergunta no *Workspace A* e garantir, por via de asserções de código, que o *Workspace B* não teve acesso a nenhuma daquelas informações ou vetores.

3. Bloqueio no Pipeline (Quality Gates): Integrar estes novos testes ao fluxo do GitHub Actions. Se qualquer alteração futura quebrar a capacidade do sistema de processar um ficheiro ou responder a um chat simulado, o pipeline deve falhar imediatamente, impedindo que a atualização seja publicada com bugs para os utilizadores finais.

Essas recomendações não anulam a necessidade de implementar a medição de cobertura sugerida na seção 3.4; pelo contrário, as abordagens se complementam. Enquanto a métrica de cobertura expõe matematicamente onde estão as lacunas, a análise baseada em riscos orienta o time sobre qual dessas lacunas deve ser priorizada. Trabalhando em conjunto, ambas as estratégias oferecem ao *AnythingLLM* um caminho realista de evolução: permitem transformar uma suíte de testes que hoje é tecnicamente bem estruturada, mas muito restrita, em uma verdadeira rede de segurança. Com isso, o projeto passa a blindar seus fluxos centrais contra erros de regressão e a proteger o que há de mais crítico: a privacidade dos dados do usuário e a estabilidade das integrações externas, que representam, simultaneamente, o grande diferencial do produto e a sua maior fragilidade.

6. PLANO DE EVOLUÇÃO DA QUALIDADE

Com base no diagnóstico consolidado nas seções anteriores, propõe-se a seguir um plano de evolução realista e priorizado, que parte da premissa de que a maturidade em testes não se constrói de uma vez, mas de forma incremental e direcionada pelo risco real do sistema.

6.1. Top 3 prioridades de teste

1. Testes de contrato para os provedores de LLM e bancos de dados vetoriais: é a lacuna de maior impacto, pois protege diretamente a promessa central do produto ("conecte qualquer modelo, qualquer banco vetorial") e mitiga o risco hoje classificado como Alto sobre o `getLLMProvider` e os 8 provedores vetoriais sem nenhuma cobertura.

2. Testes de integração para o isolamento de dados entre workspaces: trata-se de um risco de privacidade e segurança, não apenas funcional, e por isso deve ser tratado com urgência equivalente.

3. Testes de caracterização para os módulos identificados como God Object/God Switch (`files/index.js` e `getLLMProvider`): pré-requisito de segurança para que as refatorações já recomendadas na Atividade 2 possam ser executadas sem operar "no escuro".

6.2. Camadas de foco

A suíte atual já possui uma base sólida de testes unitários (camada de baixo nível), mas carece completamente da camada intermediária de testes de integração e de testes de contrato com serviços externos. O investimento inicial deve, portanto, se concentrar nessa camada intermediária — não porque os testes unitários deixaram de importar, mas porque é exatamente o "meio da pirâmide" que está ausente e que oferece a melhor relação custo-benefício: testes de integração e de contrato capturam quebras reais de comportamento entre módulos sem o custo de manutenção elevado de uma suíte E2E completa. Testes end-to-end devem ser tratados como uma camada de maturidade posterior, reservada aos fluxos mais críticos (criação de workspace → upload → chat), e não como prioridade imediata.

6.3. Implantação gradual

Para não interromper o ritmo de contribuições de uma comunidade com mais de 195 colaboradores, recomenda-se uma adoção em fases:

- Fase 1 (curto prazo, baixo atrito): ativar a flag `--coverage` no Jest e subir os relatórios para o Codecov, apenas para gerar visibilidade — sem bloquear merges ainda.
- Fase 2 (médio prazo): aplicar a regra do "boy scout" — todo PR que altera um arquivo sem teste é obrigado a adicionar cobertura mínima para a função especificamente modificada, começando pelos módulos de maior risco combinado (`stream.js`, `getLLMProvider`).
- Fase 3 (médio-longo prazo): introduzir testes de contrato com mocks para os provedores de LLM e bancos vetoriais, reaproveitando o padrão de fixtures literais já bem executado em `pgvector/index.test.js`.
- Fase 4 (longo prazo): ativar `coverageThreshold` como quality gate obrigatório e iniciar a construção da suíte de integração/E2E para os fluxos de workspace e RAG.

Essa progressão evita big-bangs: cada fase entrega valor isolado e pode conviver com o desenvolvimento ativo de novas features.

6.4. Ferramentas, pipeline e política mínima de execução

Ferramentas sugeridas: manter o Jest como base (já maduro no projeto); adotar `jest.mock/nock` para interceptação de chamadas HTTP a provedores de LLM e bancos vetoriais; avaliar Playwright ou Cypress para a futura camada E2E; usar Codecov para visualização de cobertura, por já oferecer plano gratuito para repositórios públicos.

Pipeline: estender o workflow `run-tests.yaml` já existente, adicionando um passo de upload de cobertura após `yarn test`, e criar um gate de bloqueio de merge condicionado tanto ao `coverageThreshold` quanto à presença de arquivos de teste correspondentes a alterações em `server/utils/` e `server/models/`.

Política mínima: nenhum PR que modifique lógica de negócio em módulos críticos (providers, chats, MCP, workspaces) deve ser mesclado sem teste associado — tornando tecnicamente verificável a regra que já consta, mas não é aplicada, no CONTRIBUTING.md.

6.5. Critérios de sucesso

Critério de entrada: existência de infraestrutura mínima de mocking validada (ex.: primeiro teste de contrato funcionando localmente e em CI antes de exigir o padrão de todos os PRs).

Critérios de saída:

- Existência de cobertura mensurável (não apenas pass/fail) visível publicamente via badge;
- Pelo menos um teste de contrato funcionando para cada categoria de dependência externa (LLM e banco vetorial), e não mais apenas para pgvector;
- Zero PRs de funcionalidade nova mesclados sem teste correspondente, verificável retroativamente nos últimos N PRs após a implantação do gate;
- Existência de ao menos um teste de integração validando o isolamento entre workspaces, dado que esse é o risco de maior sensibilidade identificado na auditoria;
- Redução mensurável do tempo entre a introdução de uma regressão e sua detecção (hoje, no caso do MCP, a detecção só ocorre em produção via Issues de usuários).

A maturidade, portanto, não deve ser medida apenas por um número percentual de cobertura, mas pela cobertura efetiva das áreas de maior risco combinado mapeadas nesta auditoria — é esse alinhamento entre métrica e criticidade real que transforma a suíte de testes do AnythingLLM de uma validação pontual de funções utilitárias em uma verdadeira rede de proteção para o produto.

7. REFERÊNCIAS

MINTPLEX LABS. *AnythingLLM*. Disponível em:

<https://github.com/Mintplex-Labs/anything-llm>. Acesso em: 27 abr. 2026.

DEEPWIKI. *DeepWiki - AnythingLLM*. Disponível em:

<https://deepwiki.com/Mintplex-Labs/anything-llm>. Acesso em: 27 abr. 2026.

IAs utilizadas: <https://gemini.google.com/>, <https://claude.ai>